# Unit Testing / Self-Test Report

**Dev task:** OCW-24375 — V4 bulk payment and the single-check mail / email / print and Payments Out print flows moved from synchronous per-payment sends onto the asynchronous payment-batch APIs (batch jobs + Pusher live updates + progress modal, toast, strip and details/retry surfaces)

**Type:** Frontend

**Environment:** branch `OCW-24375-SHABEEB` @ `a7b32ae12a`, baseline `origin/staging` @ `d2ba566928` (which is also the merge-base, so the branch is up to date with staging), local worktree `C:\newFile\afterBreak\OCW\wt-one`, `--configuration development`.

**No backend with the batch endpoints was reachable for this report and nothing was driven in a browser** — a browser run needs real payments submitted and real checks mailed on staging, which is a side-effectful step and was not taken unilaterally.

**Scope measured:** `git diff origin/staging...HEAD -- src` → **74 files, 28 added / 46 modified / 0 deleted, +11,353 / −1,371**. (64 non-spec files, of which 61 are lintable `.ts` / `.html`.)

**Supersedes** the 2026-10-21 version of this report, which was taken at `7d3ad0c3c9`. Since then the branch merged staging twice and added, among other things: per-row and per-section **selection** (`selected` control + `v4-bulk-section` tick-all), the `unqueued-groups.util.ts` reporting path, a retry path on the progress modal, a synchronous/batch rail split in the progress modal, a `message` input on the shared `loading-overlay-v4`, and a **move of the whole payment-batch feature out of `src/app/shared/` into `src/app/modules/v4/payment-batch/`**. Every check below was re-executed against `a7b32ae12a`; nothing was carried over.

---

## ⚠️ Read this before trusting any row below

**This repo still cannot run unit tests, and still cannot type-check them.** Executed this session:

| Command | Result |
|---|---|
| `npm test` (→ `ng test`) | **exit 1** — `Cannot determine project or target for command.` No `test` target in `angular.json`. |
| `npx tsc -p tsconfig.spec.json --noEmit` | **exit 2, 1,274 errors across 152 files** — e.g. `TS2593: Cannot find name 'describe'`, `TS2304: Cannot find name 'it'`. Repo-wide, from missing jasmine types. |
| `.eslintignore` line 8 | `*.spec.ts` — every spec is lint-exempt repo-wide |

So the **10 spec files this ticket adds — 166 `it()` blocks, 2,973 lines — have never been run and have never been linted.** They are source text that encodes intent. Do not read a spec's existence anywhere below as evidence.

One thing *is* now established about them that was not last time: of the 1,274 spec type errors, **592 fall in this ticket's spec files, and every single one is a missing-jasmine-global error.** Filtering out `Cannot find name 'describe'|'it'|'beforeEach'|'afterEach'|'expect'|'spyOn'|'jasmine'| 'fail'|'beforeAll'|'afterAll'` leaves **zero** errors in these files — no bad imports, no wrong types, no stale symbols. That is evidence they would compile the day a runner exists. It is **not** evidence they pass.

| Spec file | it() | Lines |
|---|---|---|
| `v4-check-batch-modal.component.spec.ts` | 32 | 679 |
| `v4-batch-details-modal.component.spec.ts` | 26 | 459 |
| `v4-batch-toast.service.spec.ts` | 26 | 391 |
| `v4-payment-batch.service.spec.ts` | 18 | 444 |
| `v4-bulk-section.component.spec.ts` | 17 | 199 |
| `v4-bulk-progress-modal.component.spec.ts` | 14 | 265 |
| `v4-payment-batch-strip.component.spec.ts` | 12 | 259 |
| `unqueued-groups.util.spec.ts` | 8 | 118 |
| `v4-batch-toast.component.spec.ts` | 7 | 119 |

| Spec file | it() | Lines |
|---|---|---|
| v4-payments-table.component.spec.ts | 6 | 40 |
| **Total** | **166** | **2,973** |

The checks that *were* executed here: **whole-app type-check, two dev builds, ESLint measured against an `origin/staging` baseline, an eager-bundle-graph walk, and targeted source greps.** Everything behavioural is `NOT RUN`.

---

## Summary of changes tested

1. **Bulk modal.** `processPayments()` creates one parent batch (`POST /zil-payments/batch-jobs`), then fires **up to seven** queued calls concurrently through `forkJoin` with that `batchId` merged into each body — check mail, check email, check DD, print paper type 4, print paper type 6, wallet `multi-process`, and one merged `process-payments`. The synchronous rails (`sendSequentialRows`) run alongside. One read after the burst, then Pusher.
2. **One bulk zil call.** The per-payment `/zil-payments/{uuid}/process-payment` rails collapse into a single `POST /zil-payments/process-payments` carrying ACH, wire, wallet check email, check mail type 2 (non-wallet) and let-vendor-decide rows together. `from === 'CARD'` and Virtual Card keep the per-payment rail. `isTransferTypeEnabled` gates **per kind**, so a disabled kind drops out and the rest still go.
3. **Transport is Pusher, not polling.** Channel `payment-batch-{encrypted_id}`, `share()`d so one subscription serves every consumer; the broadcast carries the whole batch. `resync$` fires on the second and later `subscription_succeeded` and each surface answers with one re-read.
4. **Five surfaces:** in-flow progress modal (now split into synchronous and batch rail blocks, with per-rail download, retry, and continue-in-background), global batch toast (mounted once in `content-layout-v4`, refcounted suppression via `watchBatch`), page-level batch strip on the Payments Out detailed tab, batch details modal with per-job download and confirm-gated retry, and the shared two-phase check-batch dialog.
5. **Selection.** Every rail row carries a `selected` form control (default true) and each `v4-bulk-section` header ticks/unticks its whole group; totals, the readiness gate and the progress modal's expected total are all selection-aware.
6. **Unqueued reporting.** `unqueued-groups.util.ts` builds a card for every group whose queue call never landed, so a user told "0 of 3 sent" is given the API's own reason instead of a blank summary. Pure function, excluded from double-reporting by uuid and by `${type}:${source}`.
7. **Issue badges.** `row-issue.util.ts` holds `hasRowIssue` / `rowIssueMessages` / `rowIssueText` shared by the section badge and the row icons.
8. **Single-check and Payments Out flows** (`CHECK_BATCH_MIN_COUNT = 2`) create their own parent batch and hand it to the check-batch dialog; below the threshold and for scheduled/recurring sends the original plain confirm is kept, and every premature "sent successfully" alert is suppressed on a batched path.
9. **Shared `loading-overlay-v4` gained an optional `message` input** — additive, the no-message branch renders the previous markup verbatim.
10. **Five existing wrappers plus four `check.service` methods gained an optional trailing `batchId`**, merged as `{ ...payload, ...(batchId && { batchId }) }`. No existing caller passes it.

---

## 1. Functional validation — NOT RUN

Nothing in this section was executed. No backend with the batch endpoints, no test runner, no browser run.

| # | Test | Expected | Actual (evidence) | Result |
|---|---|---|---|---|
| 1.1 | Mixed bulk send (check mail + email + DD + print type 4 + print type 6 + wallet mail + wallet print + ACH + wire) | One `POST batch-jobs`, then **seven** group calls fired **concurrently**, each returning immediately with the batch id | — not driven | NOT RUN |

| # | Test | Expected | Actual (evidence) | Result |
|---|---|---|---|---|
| 1.2 | ACH + wire + let-vendor-decide + wallet check email + check mail type 2 in one send | **Exactly one** `POST /zil-payments/process-payments`, body `{batchId, payments:[{id, payee_email, payee_bank_account_id?}]}`, `payee_bank_account_id` present only on ach/wire rows | — | NOT RUN |
| 1.3 | Print paper type 4 and type 6 in the same send | Two separate `checks/multi/print` calls, same `batchId`, different `paperType` | — | NOT RUN |
| 1.4 | Wallet mail + wallet print together | **One** `multi-process` call covering both, uuids deduped; no follow-on print call | — | NOT RUN |
| 1.5 | CARD-source row in the same send | Still `POST /zil-payments/{uuid}/card` per payment; reported by `sendSequentialRows` → `announceSyncRails()`; never reaches the batch; does not block the queued groups | — | NOT RUN |
| 1.6 | **Row selection** — untick individual rows, untick a whole section, mixed state | Section header shows "n of N selected" and an indeterminate box; totals and Payments Ready follow; only ticked rows reach any payload | Spec written (17 `it()`), **not executed — no runner** | NOT RUN |
| 1.7 | Untick every row in a rail | That rail contributes no queue call at all | — | NOT RUN |
| 1.8 | Pusher live update | A `payment-batch-{id}` event advances the progress modal with no HTTP read; `auditTime` collapses repeat ticks | — | NOT RUN |
| 1.9 | Socket drop and recovery | `resync$` fires on the *second* `subscription_succeeded`; each open surface does exactly **one** re-read | — | NOT RUN |
| 1.10 | Per-rail status badges | `queued \| sending \| ready \| sent \| partial \| failed`; `ready` renders for a print job whose file exists but which was not posted | — | NOT RUN |
| 1.11 | **Progress modal sync/batch split** | Synchronous rails (card, virtual card) report their own block with their own totals and percent, independently of the batch block | Spec written (14 `it()`), **not executed** | NOT RUN |
| 1.12 | **Retry from the progress modal** | `forkJoin([batchRetry, sendSequentialRows(failedSyncRails)])` — both halves retried, modal reopens from settled | — | NOT RUN |
| 1.13 | **Unqueued-group cards** | A group whose queue call 422'd gets its own card carrying the API's wording; a group the batch *does* own is never drawn twice | Spec written (8 `it()`), **not executed** | NOT RUN |
| 1.14 | Continue in background | Progress modal closes, strip picks the batch up, toast announces on completion | — | NOT RUN |
| 1.15 | Toast suppression | Silent while the progress modal or a check-batch dialog is reporting batch X; the server's completion event announces regardless; the strip never claims a watch | — | NOT RUN |
| 1.16 | Batch strip | `listBatches(20)` once on the `all/check/ach/wire` filters only; a listed batch replaced in place, an unlisted one prepended; collapsed = one summed bar | — | NOT RUN |
| 1.17 | Details modal | One read for first paint, then live updates; a later empty read never blanks what the channel delivered | — | NOT RUN |
| 1.18 | Retry from the details modal | Offered only when `batchNeedsAttention`, confirm-gated, `POST batch-jobs/{id}/retry`, reopens a settled batch (`isStaleBatchEvent` lets a retry through) | — | NOT RUN |
| 1.19 | Per-sub-batch download | `GET batch-jobs/{batchId}/download?jobId=…` re-fetched on every click (30-minute link), separately for type-4 and type-6 print jobs | — | NOT RUN |

| # | Test | Expected | Actual (evidence) | Result |
|---|---|---|---|---|
| 1.20 | Check email ≥ 2 checks | Own parent batch, dialog in `confirm` → `jobs` phase, `POST /email-checks/` carrying the batch id, **no** "sent successfully" alert | — | NOT RUN |
| 1.21 | Check mail ≥ 2 checks | `POST /cheques/sent-mail/{type}` with the batch id on **all** call sites including the retry paths; only the **ticked** rows | — | NOT RUN |
| 1.22 | Payments Out print from both tables | `v4-check-list` → `printMultiCheck`; `v4-payments-table` → `checkPrintPayment` for bank **and** wallet sources, both batched | — | NOT RUN |
| 1.23 | `selectedPrintTotal` | Sums the selection; returns `undefined` if **any** row fails to parse, dropping the amount tile; wallet path shows no amount or source tile | Spec written (6 `it()`), **not executed** | NOT RUN |
| 1.24 | Below the threshold (1 check) | `CHECK_BATCH_MIN_COUNT = 2`, so no batch, original plain confirm, server answers inline with a url | — | NOT RUN |
| 1.25 | Scheduled / recurring send | Keeps the original non-batched confirm | — | NOT RUN |
| 1.26 | ABC-legal enterprise print | Short-circuited before the dialog, stays per-check and un-batched | — | NOT RUN |
| 1.27 | Issue badge behaviour | Red `"{n} issue(s)"` pill with a per-reason tooltip; self-expands once when the count first becomes non-zero; a manual collapse sticks | Spec written, **not executed** | NOT RUN |
| 1.28 | **`loading-overlay-v4` with and without `message`** | With a message: card + 20px spinner + `role="status"`/`aria-live`. Without: the previous bare 48px spinner, unchanged | — | NOT RUN |
| 1.29 | Company switch mid-session | The channel re-keys off the new `encrypted_id` and the toast is cleared | — | NOT RUN |

**Who should run these:** QA, or the developer against a staging backend that has the batch endpoints. 1.6, 1.11, 1.13, 1.23 and 1.27 become runnable the day a `test` target exists.

## 2. Edge cases and error scenarios — NOT RUN

| Case | Expected | Actual (evidence) | Result |
|---|---|---|---|
| `batch-jobs` POST fails or returns no id | Nothing is sent; `runError` set; every summary row stamped failed; alert shown; no blank summary; the synchronous rails are not sent either | — | NOT RUN |
| One group's queue call 422s | `queued()` maps it to `false`; only that group's rows are marked failed; `forkJoin` is not aborted and the other six still queue; the group then surfaces as an **unqueued card** | — | NOT RUN |
| Every queue call rejected | `accepted.some(Boolean)` is false → no `readBatch()`, no tracking started | — | NOT RUN |
| A disabled transfer type (e.g. ACH off) | That kind drops out of the merged `payments[]`, each of its rows gets `"{Type} transfer is not enabled."`, the remaining kinds still go out in the one call | — | NOT RUN |
| A group settles `partial` | Every row of that group reports failed with `"{n} of {total} failed — see Payments Out"` (deliberate over-reporting) | — | NOT RUN |
| Non-unique `job.id` (ACH + let-vendor-decide share one) | Everything keys off `` `${job.id}:${job.type}` `` | Spec written, **not executed** | NOT RUN |
| Out-of-order Pusher event | `isStaleBatchEvent` drops it — but a **retried** batch is allowed to reopen from settled | Spec written (26 `it()` in the toast service spec), **not executed** | NOT RUN |

| Case | Expected | Actual (evidence) | Result |
|---|---|---|---|
| All-failed batch | `batchBarSegments` never renders a full green bar; `batchAttentionCount` falls back to `max(failed, total − succeeded)` | Spec written, **not executed** | NOT RUN |
| **Unticked rows vs the batch record** | `createBatch()` is still passed `bulkPaymentsUuidList` / `bulkPaymentsCheckIdList` **in full**, so a batch can be registered against payments that unticking excluded from every queue call. Known, marked `ponytail:` at `v4-bulk-payment-modal.component.ts:1313`, with the reason it cannot be narrowed yet (the four bank check rails hold `transfer.id`, a different id space). The user-visible total is selection-aware; the batch record is wider than the run. | Accepted limitation, documented | NOT RUN |
| Terminal event missed while the socket was down | **Never announced** — Pusher does not replay and no catch-up read is done. Known, marked `ponytail:` in `v4-batch-toast.service.ts`. What is on screen stays correct because the strip and dialogs re-read on `resync$`. | Accepted limitation | NOT RUN |
| A group refused while its siblings queued | Reported, but **not retried** — retry covers the batch, not a call that never created a job. Known, marked `ponytail:`. | Accepted limitation | NOT RUN |
| F5 mid-run | The bulk modal's batch id is in memory → *its* view is orphaned (payments still complete server-side); the channel is company-wide so the strip and toast do pick the batch back up. Marked `ponytail:`. | Accepted limitation | NOT RUN |
| Declined retry / unconfirmed Plaid account / non-retryable error | Each dead end closes the check-batch dialog explicitly | — | NOT RUN |
| Download clicked before the file exists | No `download` on the job → no button offered | — | NOT RUN |
| Two downloads at once in the details modal | One shared `downloading` flag disables **every** row's button while any one is in flight. Known, marked `ponytail:`. | Accepted limitation | NOT RUN |

## 3. Security — PARTIAL (static review only; behaviour NOT RUN)

- **Input validation:** unchanged. The submit guards in `onSubmit()` (form invalid, send-to-one-address email, invalid check-mail addresses) were not touched. This ticket adds **no new validation**, so there is no "records that were valid yesterday" class of regression. Selection narrows what is sent; it never widens it.
- **Authentication / authorization:** no new route and no guard change — the new surfaces live inside already-guarded v4 pages and dialogs. Every new request goes through `HttpClient` and therefore the existing interceptor, so the `CompanyID` header is attached and tenancy stays server-enforced.
  - `GET batch-jobs/{id}`, `GET batch-jobs/{id}/download`, `POST batch-jobs/{id}/retry` and `GET batch-jobs?perPage=20` all take or return a client-held encrypted batch id. **The backend must scope all four to the logged-in company.** The frontend treats the id as an opaque request parameter and never as proof of ownership. **Cross-company behaviour is NOT RUN** — it needs a two-company fixture; handed to QA.
  - The Pusher channel is keyed off `headerService._headerData.userInfo.encrypted_id`, i.e. a **company-wide** channel. Any session for that company receives every batch event for it. That matches the existing tenancy model (a company is a `users` row), but **the channel name is client-derived, so the backend must authorise the subscription** rather than trust the name.
- **Sensitive data exposure:** the print url is short-lived (30 min) and re-fetched per click rather than cached or logged. No payment data is written to `localStorage`. No `console.*` was added — see §10 for the evidence.
- **Injection / XSS:** no new template interpolation of raw API data beyond counts, statuses and existing labels. The composed strings are built from numeric counts and from `ValidationService.getValidationErrorMessage()` output, which is app-owned text, not API text. The one place API wording is surfaced verbatim is the unqueued-group card's `messages[]`, rendered as interpolated text (Angular escapes it), never as `innerHTML`.

- **Client-side only?** Three behaviours are **UX only** and must still be enforced server-side: the per-kind `isTransferTypeEnabled` gate, `CHECK_BATCH_MIN_COUNT = 2`, and **row selection** — the backend must reject a disabled transfer type on `process-payments`, must decide for itself whether a send is queueable, and must treat the ids in each queue call as authoritative rather than the wider id list `createBatch` registered.

## 4. Performance and optimization — PARTIAL

- **Request count (the point of the ticket).** A send goes from *N serial* calls (one per payment across six rails) to *1 + up to 7 concurrent* calls, plus **one** read after the burst. Steady state costs **zero** requests: the Pusher broadcast carries the whole batch, `jobs[].payments[]` included. **The latency improvement itself is NOT RUN** — it needs a real backend.
- **Nothing polls — PASS (verified by grep, not by behaviour).** Executed: `grep -rE '\b(setInterval|interval\(|timer\(|retry\(|retryWhen|repeat\()'` across `src/app/modules/v4/payment-batch/` and `src/app/modules/v4-bulk-payment/modals/v4-bulk-payment-modal/`, excluding specs. Matches: exactly two, both the **method** `retry(): void` on the details modal and the check-batch modal — not the RxJS operator. `setTimeout` appears twice in ticket code, both in `v4-batch-toast.service.ts` (`private timers = new Map<…>()` and `setTimeout(() => this.dismiss(id), timeout)`, the toast auto-dismiss); the third `setTimeout` hit, in `v4-check-mail-type2-bulk.component.ts:316`, is **pre-existing** (confirmed: no `+/-` line for it in `git diff origin/staging...HEAD` on that file). No `timer`, no `interval`, no tick cap anywhere.
- **One shared subscription.** `channel$` is `share()`-refcounted and lazy, so injecting `V4PaymentBatchService` connects nothing until a consumer subscribes. Verified by reading the source; **not** verified at runtime (no socket count observed) → that half is NOT RUN.
- **Bundle impact — PASS (executed).** Walked the eager graph from `index.html`'s three entry scripts (`polyfills.js`, `scripts.js`, `main.js`) following only **static** `from "./x.js"` / `import "./x.js"` edges: **108 files**. Grepped that graph for symbols and strings unique to this work:

| Needle | In eager graph? |
|---|---|
| `payment-batch-` (the channel prefix) | **No** |
| `process-payments` | **No** |
| `V4BatchToastService` | **No** |
| `unqueuedGroups` | **No** |
| `hasRowIssue` | **No** |
| `LoadingOverlayV4` | **No** |
| `Continue in background` / `Payments Ready` / `issue(s)` | **No** |
| `batch-jobs`, `V4PaymentBatchService` | **Yes — in `chunk-DCPVE3S2.js`, and only inside the JSDoc comments added to `check.service.ts`** (verified by printing the surrounding 160 characters: both hits are `* @param batchId - Parent payment batch id, from \` POST /zil-payments/batch-jobs\`)`. check.service.ts\` is already eager on staging; no batch **code** is pulled in, and a production build strips comments. |

Harness validated rather than assumed: `payment-batch-` **does** exist in the build output, in three lazy chunks, `Continue in background` in one, and `ContentLayoutV4` is **absent** from `main.js` — the whole v4 shell is lazy, so mounting `<app-v4-batch-toast />` in it costs the initial bundle nothing. Initial total `20.33 MB` (dev build, unminified) — unchanged from the previous report's measurement.

- **Rendering / change detection:** the new surfaces are signal-driven; `auditTime` on `batchUpdates$` in the bulk modal collapses the identical repeat ticks the channel sends. Row selection is a form control rather than a side-car `Set` specifically so toggling rides the existing `valueChanges` subscription instead of adding one. Not profiled → NOT RUN.

- **Subscriptions:** reviewed in code, not executed.
  - The submit subscription is deliberately **not** in `this.subscriptions`, so the run survives the modal closing. Its `watchBatch` claim is guarded on `isDestroyed`, so a dialog closed before the batch id landed cannot claim a watch no `ngOnDestroy` can release.
  - `V4CheckBatchModalComponent` releases its `watchBatch` claim on destroy; that teardown is what makes "run in background" hand off cleanly.
  - **Known, deliberate:** `V4BatchToastService` subscribes in its constructor and binds to the **root** `DestroyRef`, so a session that visits a V4 page and then moves to V3 keeps the channel open for the rest of the session. No UI, no requests, no V3 behaviour change — but it is the one subscription in this ticket with session lifetime.

## 5. Code quality — PASS (after fixing one defect found here)

**ESLint, measured against an `origin/staging` baseline.** Executed: lint all 61 non-spec `.ts`/`.html` files in the ticket diff; then `git checkout origin/staging -- <the 12 modified files that report anything>`, lint again, `git checkout HEAD -- <same>`, and confirm the tree was restored (`git status --porcelain -- src` showed only the intended a11y fix below, nothing else).

| File | Baseline (`origin/staging`) | After |
|---|---|---|
| `check.service.ts` | 129 err | 129 err |
| `v4-bulk-payment-modal.component.ts` | 33 err | **19 err** |
| `check/model/check.ts` | 12 err | 12 err |
| `v4-group-payment.component.ts` | 7 err | 7 err |
| `v4-payment-summary-modal.component.ts` | 5 err | 5 err |
| `v4-check-mail-bulk.component.ts` | 3 err, 1 warn | 3 err, 1 warn |
| `v4-check-email-bulk.component.ts` | 4 err | 4 err |
| `v4-ach-bulk` / `v4-check-dd-bulk` / `v4-check-mail-type2-bulk` `.component.ts` | 2 err each | 2 err each |
| `v4-group-payment.component.html` | 1 err | 1 err |
| `v4-bulk-payment.service.ts` | 1 err | 1 err |
| **All 17 new lintable non-spec files** (model, both services, the four `v4-*batch*` surfaces, progress modal, bulk section, both utils) | — | **0 err, 0 warn** |
| **Total** | **201 err, 1 warn** | **187 err, 1 warn** |

**No new violations remain; fourteen fewer on the modal; every new file clean.** The remaining counts are pre-existing (`no-explicit-any` in the v3 models/services, one `import/no-cycle` pair, `import/order`, `no-restricted-globals` on `open`). No rule was disabled inline. These totals are **not** a clean-lint claim — this repo is far from lint-clean; the gate used is "at or below baseline, per file".

- **One defect found and fixed here** — see the defects table: `v4-bulk-section.component.html` (a new file) was shipping **2 new a11y errors**. Fixed and re-verified.
- **Follows existing patterns:** repository-pattern service per the module folder split, standalone components, `inject()`, signals, `@if`/`@for`, Tailwind utilities with project colour variables, Lucide icons. `BATCH_GROUP_SUMMARY_TYPE`, the display helpers, `hasRowIssue` and `unqueuedGroups` exist specifically so no two surfaces can draw the same batch differently.
- **Structure improved since the last report:** the payment-batch feature moved out of `src/app/shared/` into `src/app/modules/v4/payment-batch/{model,services,components,modals}`, which is the folder split `CLAUDE.md` asks for. `unqueued-groups.util.ts` was extracted as a pure function explicitly so it can be checked without standing up the modal.
- **Dead code removed (orphaned by this change):** `V4BulkPaymentService`'s own `processAllPaymentsSequentially()` and its `completedPaymentCount` signal, plus the orphaned `from`/`concatMap`/`finalize`/`tap`/`catchError` imports.

- **Net-new dead code left behind — flagged, not hidden, and still open.** `V4PaymentSummaryModal`'s `batchId` / `printDownloadAvailable` inputs, its "Download checks" button (`v4-payment-summary-modal.component.html:241-247`) and `downloadPrintFile()` were added by this ticket and orphaned by it. Verified again at `a7b32ae12a`: **both** call sites — `header-v4.component.ts:1628` and `v4-bulk-payment-modal.component.ts:2049` — pass only `{ summary }`, and the bulk modal's doc comment now states the omission is deliberate ("no `batchId`: there is no print file on a card rail"). So `canDownloadPrint` is permanently false and the button can never render. **This should be wired or deleted before merge.**
- **Pre-existing debt flagged, not touched:** grouping logic derived more than once; `CheckPrintFormBody.checkId` typed `number` while holding the encrypted string id (coerced at the call site); the `import/no-cycle` pair between the modal and `v4-bulk-payment.service.ts`.

## 6. Error handling and logging — PARTIAL (static review only)

- Every queue call is wrapped by `queued()` → maps to `true`, catches to `false` and records the failure against that group's rows. One group failing to enqueue cannot abort the `forkJoin`.
- A failed `createBatch()` sets `runError`, stamps every summary row failed and raises an alert — the summary is never blank, and the synchronous rails are not sent either.
- A group that never queued is now **explained**, not just counted: `unqueuedGroups()` surfaces the API's own wording, falling back to `'Processing failed'` only when the row carried no reason.
- A flagged row always has a reason: `rowIssueMessages()` falls back to `'This payment needs attention before it can be sent.'` when nothing could be named.
- `isDisable422: 'true'` is set on every new request so the documented 422 bodies arrive as data instead of raising a global warning alert.
- Reads are **single-attempt by design** (no retry operator — see §4). Recovery is `resync$` plus the manual refresh buttons on the progress modal and the strip, plus explicit retry on the progress modal and the details modal.
- `queuedBatch` on `IProcessCheck` is what stops the chained process-loading modal announcing a queued send as finished; every "sent successfully" alert on a batched path is suppressed.
- **No `console.*` / `debugger` / `alert` added.** Evidence: `git diff origin/staging...HEAD -- src | grep -E '^\+.*(console\.|debugger|\balert\()'` → **zero added lines**; and `no-console`, `no-debugger`, `no-alert` are hard errors in this config, with the 17 new files at 0 errors and every modified file identical to its baseline (§5).
- **The behaviour of all of the above is NOT RUN** — reviewed in code and linted, never executed.

## 7. Database migrations and rollback — N/A

Frontend-only change. No migrations, no schema, no seed data.

## 8. API backward compatibility — PASS (compile-time, whole app) / NOT RUN (runtime)

- **Contracts changed:** none broken. `batchId` is an **optional trailing parameter** on `sendCheckMail`, `checkPrintPayment`, `processWalletPaymentsMulti` (`v3-bulk-payment.service.ts`) and on `sendEmail`, `sendmail`, `printMultiCheck`, `postDirectDepostModalData` (`check.service.ts`). Each merges as `{ ...payload, ...(batchId && { batchId }) }` — spreading a falsy value is a specified no-op, so when the argument is omitted the key is never *created* and the body is byte-identical. Every V3 call site omits it.
- `LoadingOverlayV4Component` gained an **optional** `message = input<string>()`. Verified: it has **4 consumers**, and **exactly one** passes `[message]` (`v4-group-payment.component.html:7`). The other three hit the `@else` branch, which is the previous markup character-for-character.
- `IZilPaymentsForms` went module-private → exported (additive). `zilPayments()` and `V3BulkPaymentService.processAllPaymentsSequentially()` are **untouched**. `isTransferTypeEnabled()` **duplicates** the gate rather than extracting it, which is why the V3 path is untouched.
- **Two return types widened**, absorbed: `checkPrintPayment` / `printMultiCheck` from `IApiRes<{url: string}>` to `IApiRes<IPrintMultiCheckRes>` (`url` now optional), and `sendmail` from `IApiRes<string>` to `IApiRes<ICheckBatchStartRes>`. Nothing broke at compile time — the V3 print consumers carry a pre-existing

`as IApiRes<{url: string}>` cast, the uncast sites feed `window.open`, which accepts `undefined`, and every V3 `sendmail` consumer reads only `res.success`.

- **New endpoints consumed:** `POST /zil-payments/batch-jobs`, `GET /zil-payments/batch-jobs/{id}`, `GET /zil-payments/batch-jobs?perPage=20`, `GET /zil-payments/batch-jobs/{id}/download?jobId=`, `POST /zil-payments/batch-jobs/{id}/retry`, `POST /zil-payments/process-payments`.
- Compile-time compatibility is confirmed across the **whole app** by `tsc -p tsconfig.app.json --noEmit` exit 0 and a clean `ng build` (which also type-checks every template). **Runtime confirmation is NOT RUN.**

> ⚠️ **Open question for the API owner — still open, still the one release risk in this ticket.** Re-read at `a7b32ae12a`: the doc comments in `check.service.ts` still say "*More than one check is queued and answers with the batch … instead of a url*" while each `@param batchId` still says "*omit it to leave the server to decide as before.*" Those contradict each other. **If queueing is triggered by check count rather than by the presence of `batchId`**, then V3 bulk print — multi-check by definition — gets no `url`, and `v3-bulk-payment.component.ts:905,965` plus `v3-payments-table.component.ts:1498,1507,1518,2844,2857` would `window.open(undefined)` (blank tab, no PDF) while `res.success` is true, firing the GTM event and completing the progress bar; V3 `sendmail` callers would likewise report "Mail Sent successfully" over queued-only work. If queueing is gated strictly on the `batchId` field — which only V4 paths send — V3 is clean and the optional `url` is cosmetic. **One network-tab check on a V3 two-check bulk print settles it. Confirm before release.** The pre-existing casts above would then *lie* without the compiler warning.

## 9. UI responsiveness and browser/device compatibility — NOT RUN

- **Desktop:** not driven. No backend with the batch endpoints, so none of the new surfaces could be rendered in a real state.
- **Mobile breakpoints:** not driven.
- **Browsers:** none.

**Limitation named:** this ticket adds **five** new rendered surfaces — progress modal, toast, strip, details modal, check-batch dialog — plus per-row and per-section checkboxes and issue badges on 11 rail cards, a rebuilt `v4-group-payment` summary footer, and a new card variant of the shared `loading-overlay-v4`. The bulk modal dialog was also re-sized from `maxWidth: 1300px` / `width: 95%` to `width/maxWidth: 80vw` with `maxHeight: 90vh`. **None of it was rendered at any viewport.** The `80vw` / `90vh` change remains the single highest-risk layout item — a different sizing model, not a tweak, and on a small screen `80vw` is narrower than the old `95%`.

**White labels:** every brand builds from this same source, so all of it ships to all of them; the payment-batch move out of `src/app/shared/` does not change that. The brand-sensitive pieces are `content-layout-v4` (now mounting the toast for every brand) and `loading-overlay-v4` (shared, 4 consumers). Only existing colour variables and existing `common-button-v4` variants were used and no component SCSS was added beyond 25 lines in `v4-bulk-section` + 4 in the modal — but **no non-default brand build was rendered**, and the two builds run here were the default configuration only.

**Accessibility:** a defect was found and fixed here (see the defects table). The remaining a11y lint error on `v4-group-payment.component.html:222` (`label-has-associated-control`) is **pre-existing** — 1 error at baseline, 1 after.

## 10. Console errors and warnings — NOT RUN at runtime

- **Errors:** not observed; the app was never served.
- **Warnings:** `ng build --configuration development` produced exactly **one** warning, `warn - The min-* and max-* variants are not supported with a screens configuration containing objects.` — a Tailwind config warning. `tailwind.config.js` is **not in this diff**, so it is pre-existing. No warning attributable to these files. That is evidence about the build only, **not** about runtime console output.
- What *is* established: no `console.*`, `debugger` or `alert` statement was added anywhere in the diff (grep evidence in §6, plus the lint evidence in §5).

## 11. Regression / no breaking impact — PARTIAL

**Verified (executed):**

- **Whole-app type-check:** `npx tsc -p tsconfig.app.json --noEmit` → **exit 0, no output**. Every caller of the seven re-signatured wrappers, the widened `loading-overlay-v4` input and both widened return types still compiles.
- **Whole-app build including template type-checking:** `npx ng build --configuration development` → **exit 0**, `Application bundle generation complete. [100.276 seconds]`. Re-run **after** the a11y fix: **exit 0**, `[93.873 seconds]`, initial total `20.33 MB`. This covers all 74 files' templates.
- **ESLint at or below baseline on every modified file** (187 vs 201 err; §5), measured against `origin/staging` with the tree confirmed restored afterwards, and **0** on all 17 new lintable files.
- **No ticket runtime code reaches the initial bundle** — eager-graph walk over 108 files, §4.
- **`loading-overlay-v4`'s default branch is byte-identical markup** and 3 of its 4 consumers take it.
- **Nothing V3-owned is in the diff.** Confirmed from `git diff origin/staging...HEAD --name-status`: no file under `v3-layout/`, `v3-payments-table/`, `check/pages/` or `v3-bulk-payment/pages/`, and `src/app/core/`, `styles.scss`, `tailwind.config.js`, `app.module.ts` and `app-routing.module.ts` are all absent — so no shared singleton, global style or route changed.
- **The branch is current with staging** — merge-base equals `origin/staging` (`d2ba566928`), so these results are not stale against an older baseline.

**Open (handed to QA):**

- **V3 bulk payment flow** — shares all seven re-signatured wrappers. Compiles, and the request bodies are provably unchanged when `batchId` is omitted, but **not exercised**. Still the highest-value regression check in the ticket, and coupled to the open API question in §8.
- **V3 bulk print / V3 `sendmail`** specifically — see the §8 warning. If the backend queues on count rather than on `batchId`, this is a live V3 breakage, not a theoretical one.
- **The three other `loading-overlay-v4` consumers** must still render the bare spinner.
- **Single-check flows below the threshold** (1 check: check view, mailing dashboard, Payments Out print of one row) must still send inline and still show their original success alert.
- **Scheduled and recurring sends** must still use the original confirm.
- **ABC-legal enterprise print** must still go per-check and un-batched, and the header's floating progress panel — which a V4 bulk run now deliberately *clears* — must still be raised by it.
- **The Payments Out detailed tab** on the filters that are *not* batch-capable (anything other than `all`/`check`/`ach`/`wire`) must render exactly as before, with no strip.
- **Pre-existing data:** this change adds **no new validation**, so there is no "valid yesterday, invalid today" risk. The closest equivalent is an in-flight batch, which cannot exist before deploy.

## 12. Acceptance criteria — NOT RUN

| AC | Result (evidence) |
|---|---|
| A multi-payment send issues one parent batch call plus one queued call per group, not one call per payment | **NOT RUN** — implemented and reviewed; needs a backend to observe |
| All queueable zil payments go out in a single `process-payments` call, Virtual Card and CARD-source excluded | **NOT RUN** |
| Queued calls return immediately and the UI tracks progress without polling | **PARTIAL** — "without polling" established by grep (§4); "tracks progress" **NOT RUN** |
| Live updates arrive over Pusher and cost zero requests in the steady state | **NOT RUN** |
| A dropped socket recovers via `resync$` with one re-read per surface | **NOT RUN** |
| Print PDFs are reachable per sub-batch from the progress modal, the toast and the strip's details modal | **NOT RUN** |

| AC | Result (evidence) |
|---|---|
| A finished batch is announced wherever the user is, and not double-announced while a surface is already reporting it | **NOT RUN** |
| A badly settled batch can be retried, from the details modal and from the progress modal | **NOT RUN** |
| A group that never queued is reported with the API's own reason rather than a blank summary | **NOT RUN** — `unqueuedGroups()` is pure and spec'd (8 `it()`), but the spec has never executed |
| The user can exclude individual payments and whole sections before sending | **NOT RUN** — spec'd (17 `it()`), never executed |
| Check email / check mail / Payments Out print of ≥ 2 checks run through the shared batch dialog and suppress the premature success alert | **NOT RUN** |
| Sends below `CHECK_BATCH_MIN_COUNT`, scheduled sends and ABC-legal enterprise print are unchanged | **NOT RUN** |
| Continue in background works and hands off cleanly | **NOT RUN** |
| Nothing in the existing V3 flow changes | **PARTIAL** — compiles, bodies provably identical, nothing V3-owned in the diff; runtime unverified and gated on the §8 question |

## Defects found during self-testing

| # | Defect | Fix | Re-verified |
|---|---|---|---|
| 1 | **Two new a11y lint errors in a brand-new file.** `v4-bulk-section.component.html:19` — the `<label>` that paints the section checkbox carries `(click)="$event.stopPropagation()"`, which tripped `@angular-eslint/template/click-events-have-key-events` and `interactive-supports-focus`. Found by the §5 baseline comparison: the file is new, so these were **net-new** violations, and `CLAUDE.md` forbids merging past a lint failure. The real interactive element is the nested `<input type="checkbox">`, which is focusable, keyboard-operable and already carries an `aria-label`; the label's handler exists only to stop the click reaching the header's collapse toggle. | **Fixed.** Added `role="presentation"` to the label — the same pattern already used on the header `<div>` eight lines above, which has a `(click)` and is not flagged. One attribute; no behaviour change. | **Yes.** `npx eslint src/app/modules/v4-bulk-payment/modals/v4-bulk-payment-modal/components/v4-bulk-section/v4-bulk-section.component.html -f compact` → **no output (0 problems)**, down from 2 errors. Full re-lint total fell 189 → 187 errors. `ng build --configuration development` re-run after the fix → **exit 0, 93.873 s**. |
| 2 | **Specs cannot be run or type-checked.** `npm test` → exit 1 (`Cannot determine project or target for command.`); `npx tsc -p tsconfig.spec.json --noEmit` → exit 2, 1,274 errors across 152 files, all from missing jasmine types. | Not fixable inside this ticket — it needs a `test` architect target in `angular.json` plus karma/jasmine (or jest). Raised as an open item below. | N/A — the finding *is* the re-verification; the command output is the evidence. **Narrowed since the last report:** all 592 errors falling in this ticket's specs are missing-jasmine-globals; filtering those leaves **zero**, so no spec has a bad import or a stale symbol. |
| 3 | **Net-new dead code still shipped by this ticket.** `V4PaymentSummaryModal`'s `batchId` / `printDownloadAvailable` inputs, its "Download checks" button and `downloadPrintFile()` were added here and orphaned here. Re-checked at `a7b32ae12a`: both call sites pass only `{ summary }`, so `canDownloadPrint` is permanently false. | **Not fixed.** Flagged in §5 and in `OCW-24375.md`. Needs a decision — wire it, or delete it. | No — open since the previous report. |

## Sign-off

| Item | Result |
|---|---|
| Build / type-check | **PASS** — `tsc -p tsconfig.app.json --noEmit` exit 0; `ng build --configuration development` exit 0 twice (100.276 s before the fix, 93.873 s after), initial total 20.33 MB, one pre-existing Tailwind warning |

| Item | Result |
|---|---|
| Spec type-check | **FAIL (pre-existing, repo-wide)** — `tsc -p tsconfig.spec.json --noEmit` exit 2, 1,274 errors, no jasmine types installed. Ticket specs contribute 592, all of them missing-global errors. |
| Unit tests | **NOT RUN** — `npm test` exit 1, no `test` target, no runner installed. **166 `it()` blocks across 10 files have never executed.** |
| Functional validation | **NOT RUN** — no backend with the batch endpoints, no browser run |
| Edge cases | **NOT RUN** — five limitations are known, deliberate and marked `ponytail:` in source |
| Security | **PARTIAL** — static review only; cross-company scoping of `batch-jobs/*`, Pusher subscription authorisation, and server-side enforcement of the selection narrowing are all **NOT RUN**, handed to QA/backend |
| Performance | **PARTIAL** — "nothing polls" verified by grep, zero ticket runtime code in the initial bundle verified by a 108-file eager-graph walk; the latency win itself **NOT RUN** |
| Code quality | **PASS** — 187 vs 201 baseline errors, no new violations remaining, all 17 new lintable files at 0, one new-file a11y defect found and fixed here |
| Error handling | **PARTIAL** — reviewed in code and linted, never executed |
| Database migrations | **N/A** — frontend only |
| API compatibility | **PASS** (compile-time, whole app) / **NOT RUN** (runtime) — and see the open §8 question |
| Responsiveness | **NOT RUN** — five new surfaces, new checkboxes, a changed dialog sizing model and a new shared-overlay variant, none rendered at any viewport, no brand build other than the default |
| Console warnings | **NOT RUN** at runtime; no `console` / `debugger` / `alert` added (grep + lint evidence) |
| Regression | **PARTIAL** — type-check, build, lint, bundle graph, consumer count and diff scope verified; every V3 and sub-threshold runtime path **NOT RUN** |
| **Ready for QA** | **No — see below** |

**Defects found during unit testing:** 3 (1 fixed and re-verified, 2 open)

**Why not ready for QA yet:** every behavioural claim in this ticket is unverified. The change rewrites the money-sending path of the V4 bulk payment modal *and* the single-check mail / email / print flows *and* Payments Out print, adds five new rendered surfaces, and adds row-level selection that decides what actually gets sent. Nothing was executed beyond compilation, linting, a bundle-graph walk and source greps. Two items should be settled before QA even starts: the §8 API question (because the answer decides whether V3 bulk print is broken today) and the net-new dead code in `V4PaymentSummaryModal`.

**Open items for QA:**

1. **Settle §8 first.** One network-tab check: V3 bulk print of two checks. Does the response carry a `url`, or a batch? If it carries a batch, V3 bulk print and V3 `sendmail` are broken on this branch and the rest of QA is moot until it is fixed.
2. Drive a mixed bulk send end to end against a backend with the batch endpoints (§1.1–1.5) and confirm in the Network tab that the group calls fire **concurrently**, and that there is **exactly one** `process-payments` call.
3. **Selection:** untick individual rows and whole sections and confirm only the ticked rows are in each payload, that totals and "Payments Ready" follow, and that an all-unticked rail produces no call. Confirm the backend ignores the wider id list `createBatch` registers (§2, the `ponytail:` note at `v4-bulk-payment-modal.component.ts:1313`).
4. Confirm live updates arrive over Pusher with **no** repeated `GET batch-jobs/{id}`, then kill the socket (devtools offline → online) and confirm `resync$` produces exactly one re-read per surface.
5. Confirm per-sub-batch download works for print type 4 **and** type 6 **and** wallet check print, from the progress modal, the toast and the details modal, including after "Continue in background".
6. Force a group's queue call to fail and confirm the **unqueued-group card** appears with the API's reason, and that a group the batch *does* own is never drawn twice.
7. Confirm retry from **both** the progress modal and the details modal reopens a settled failed batch, and that the synchronous rails are retried alongside the batch from the progress modal.
8. Confirm toast behaviour: silent while a surface is reporting that batch, sticky on failure, auto-dismiss when clean, announced by the server's completion event regardless, cleared on company switch.

9. Regression: V3 bulk payment; the three `loading-overlay-v4` consumers that pass no `message`; single-check sends below `CHECK_BATCH_MIN_COUNT`; scheduled/recurring sends; ABC-legal enterprise print (+ its header progress panel); and the non-batch-capable Payments Out filters.
10. Layout: all five new surfaces plus the section/row checkboxes at desktop and mobile widths, plus the bulk modal's new `80vw` / `90vh` sizing, plus the rebuilt `v4-group-payment` footer — and on at least one non-default white label.
11. Cross-company: confirm the backend scopes `batch-jobs/{id}`, `/download`, `/retry` and the list to the logged-in company, that a foreign uuid in `process-payments` is ignored, and that the client-derived Pusher channel name is authorised server-side.
12. **Separately, as its own ticket:** this repo cannot run *or type-check* unit tests (no `test` target in `angular.json`, no karma/jasmine/jest/vitest installed, `*.spec.ts` ESLint-ignored). Until that is fixed, the "unit testing" half of this report cannot be honoured on any ticket in this codebase, and the 2,973 lines of specs this ticket adds are documentation.

**Tested by:** Shabeeb K (with Claude Code) · **Date:** 2026-10-28